

智慧能源管理平台 部署实施方案

文档项	内容
文档名称	智慧能源管理平台 部署实施方案
系统名称	智慧能源管理平台
系统版本	V4.6.0
文档版本	V1.0
编制日期	2026-09-14
编制人	实施运维组
适用范围	V4.6.0 全新部署、版本升级部署、单机/单机 Docker 生产环境
读者对象	实施工程师、运维工程师、DBA、网络管理员、项目负责人

文档审批

角色	姓名	部门	审批意见	签字/日期
编制		实施运维组		
审核（技术）		研发组		
审核（运维）		运维组		
批准		项目负责人		
客户确认		客户方信息科		

修订记录

版本	日期	修订说明	修订人
V1.0	2026-09-14	首次发布，覆盖 V4.6.0 全新部署全流程	实施运维组

1 部署目标与范围

1.1 部署目标

在客户机房（或指定服务器）完成 V4.6.0 平台的安装部署，实现：

1. 后端服务 `inpsur-ems-admin`（Spring Boot 2.7.9 单体内嵌 Undertow）正常运行，端口 **8088**，上下文路径 **`/autoee-iot-ems`**；

2. 前端 `inspur-admin-ui` (Vue 2 + Element UI) 经 Nginx 托管, 浏览器通过 **`http://服务器 IP/`** 访问;
3. MySQL、Redis、RabbitMQ、EMQX 四类必选中间件以 Docker Compose 方式运行; TDengine 为可选时序库 (默认关闭, 按需启用);
4. 设备 MQTT 数据链路 (设备 → EMQX → 后端 → RabbitMQ/MySQL/TDengine) 打通;
5. 医院 IOT 平台回调链路 (IOT 平台 → Nginx/后端 `/hospital/callback/data`) 调通;
6. XXL-Job 调度中心、Spring Boot Admin 监控等配套组件部署到位;
7. 默认口令全部修改、端口按最小化原则开放、备份策略生效。

1.2 部署范围

范围内	范围外
后端、前端、5 类中间件、XXL-Job、Spring Boot Admin	服务器硬件采购、机房网络综合布线
数据库 <code>autoee_ems</code> 初始化、医院模块加性脚本、菜单脚本	客户侧 IOT 平台自身改造 (仅提供对接配置)
Nginx 反向代理、HTTP 访问 (HTTPS 为加固可选项)	第三方堡垒机、VPN、等保测评设备
MQTT 设备接入联调 (含 simulator 模拟)	真实电表/水表设备的现场安装与调试 (由设备厂商负责, 平台侧配合)
备份策略配置与一次恢复演练	长期容量规划与异地容灾建设

1.3 部署原则

- **只增不改**: 生产部署脚本 `docker-compose-54.yml` (192.168.8.54 现网脚本) **必须保持原样, 严禁修改**; 环境差异通过环境变量、外挂配置文件或新建 compose 文件解决。
- **配置与程序分离**: 所有环境相关参数 (IP、口令、vhost) 通过 `application-{profile}.yml`、环境变量或 Nginx 配置注入, 不将生产口令写入代码仓库。
- **先备份后变更**: 任何升级/割接操作前先完成数据库与配置备份。
- **最小权限、最小端口**: 中间件端口默认不对外, 仅 80 (及 443)、MQTT 必要端口对外开放。
- **可回退**: 每次上线保留上一版本 jar/镜像/dist 与数据库备份, 保证可在约定时间内回退。

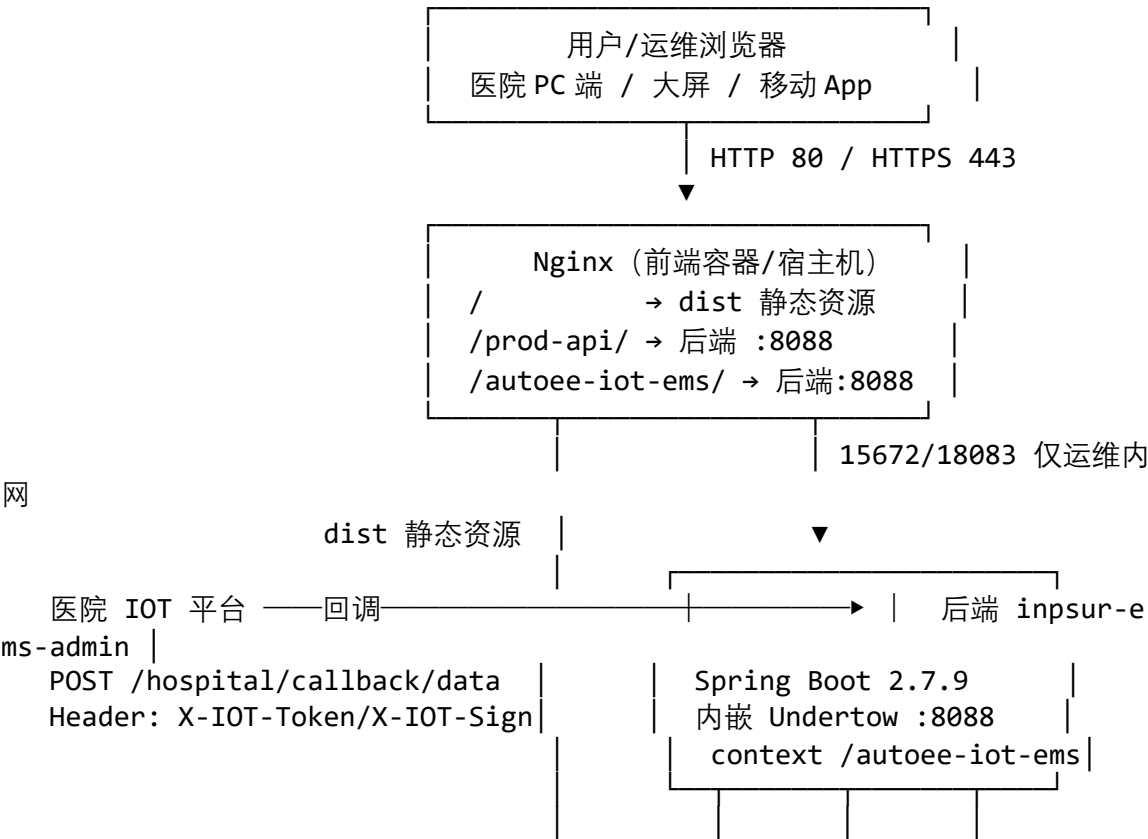
2 部署架构

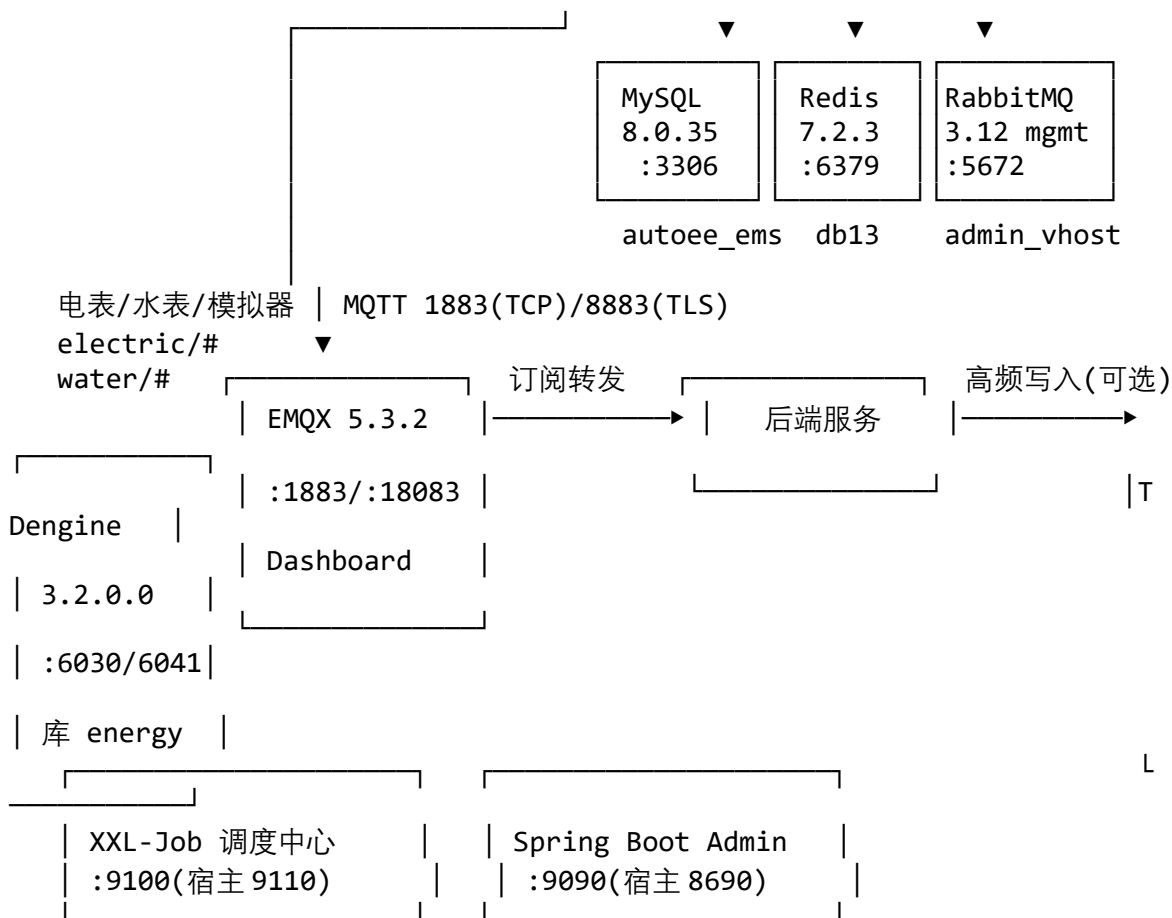
2.1 总体架构

采用前后端分离 + Nginx 反向代理 + Docker Compose 中间件架构：

- 浏览器/移动端 → Nginx (80 端口)：静态资源直接返回，/prod-api/、/autoee-iot-ems/ 反向代理至后端 8088；
- 后端为 Spring Boot 2.7.9 单体（内嵌 Undertow），对外 REST API，对内消费 MQTT/AMQP；
- 设备/模拟器 → EMQX (1883/8883) → 后端 MQTT 客户端订阅 → 业务处理 → RabbitMQ 异步解耦 → MySQL（业务库）/TDengine（高频时序，可选）；
- 医院 IOT 平台 → Nginx → 后端回调接口（Token/IP 白名单/签名鉴权）；
- 运维侧：Spring Boot Admin 监控后端 JVM，XXL-Job 调度中心负责任务调度。

2.2 ASCII 拓扑图





网络 (自研全栈 compose) : inpsur-ems-network (bridge)

现网 192.168.8.54: 复用 legal-network, MySQL/Redis 复用服务器已有容器

2.3 组件关系与版本清单

组件	版本/形态	容器名	部署方式	必选
后端	inpsur-ems V4.6.0, jar: inpsur-ems-admin.jar	inpsur-ems-backend (容器形态)	jar/systemd 或 Docker	是
前端	inpsur-admin-ui (Vue2), 构建产物 dist	inpsur-ems-frontend (容器形态)	Nginx 或 Docker	是
MySQL	mysql:8.0.35	inpsur-ems-mysql	docker-compose.yml	是
Redis	redis:7.2.3-alpine	inpsur-ems-redis	docker-compose.yml	是

组件	版本/形态	容器名	部署方式	必选
RabbitMQ	rabbitmq:3.12 - management-alpine	inpsur-ems-rabbitmq	docker-compose.yml	是
EMQX	emqx/emqx:5.3.2	inpsur-ems-emqx	docker-compose.yml	是
TDengine	tdengine/tdengine:3.2.0.0 (现网镜像; 3.3.6.13 部署要求见 5.5)	inpsur-ems-tdengine	docker-compose.yml	否
XXL-Job	inpsur-ems/xxl-job:4.6.0	inpsur-ems-xxl-job-admin	docker-compose-54.yml	是
Spring Boot Admin	inpsur-ems/monitor:4.6.0	inpsur-ems-monitor	docker-compose-54.yml	推荐

3 服务器与资源规划

3.1 硬件配置

规模	CPU	内存	系统盘	数据盘	适用场景
最低 (POC/演示)	4 核	8 GB	100 GB SSD	与系统盘同盘	单节点全容器、设备 < 100、演示
推荐 (单机房生产)	8 核	16 GB	100 GB SSD	200 GB SSD 独立挂载 /data	设备 500~2000、日消息量百万级
扩容 (多院区/高负载)	应用节点 4C8G + 中间件节点 8C16G 起	—	—	TDengine/MySQL 独立盘	设备 > 2000 或消息量 > 千万/日

容量估算参考：单条设备消息约 0.5~1 KB；仅入 MySQL 时每千万条约 5~10 GB；启用 TDengine 高频落库时按点位数 × 采集频率 × 保留天数单独测算（TDengine 压缩比通常 5~10 倍）。

3.2 软件版本要求

软件	版本要求	说明
操作系统	CentOS 7.9 / 麒麟 V10 / Ubuntu 22.04 (x86_64)	生产建议与现网一致；统一时区 Asia/Shanghai
JDK	JDK 1.8（强制，8u371 以上推荐）	编译/运行均以 1.8 为目标； 不兼容 JDK 17+ 直接启动
Maven	3.8+（构建机）	产物 inpsur-ems-admin.jar
Node.js	16/18/20（构建机；前端镜像使用 node:20）	需支持 --openssl-legacy-provider（Node 17+ 必须）
Docker Engine	24.0+	含 Docker Compose V2（docker compose）
Docker Compose	v2.20+	旧版 docker-compose v1 亦可，命令相应替换
Nginx（非容器部署时）	1.24+	容器形态镜像为 nginx:1.28.3-alpine
浏览器	Chrome 100+/Edge 100+	平台不支持 IE

JDK 兼容性强约束：若因客观条件只能使用 JDK 17/21 启动，**必须在启动命令中追加 7.2 节列出的 10 项 --add-opens 参数**，否则所有页面报“系统内部错误”（HTTP 500，异常 InaccessibleObjectException）。生产仍**强烈建议使用 JDK 1.8**。

3.3 目录规划（推荐）

```
/opt/inpsur-ems/                                # 部署根目录（属主 ems:ems）
├── backend/
│   ├── inpsur-ems-admin.jar                    # 当前版本
│   ├── inpsur-ems-admin.jar.bak                # 上一版本（回滚用）
│   ├── application-override.yml                # 环境覆盖配置（口令/IP，不入库）
│   └── logs -> /data/ems/logs/backend
├── frontend/
└── dist/                                        # 当前前端产物
```

```

├── dist.bak/                # 上一版产物
├── nginx/
│   └── conf.d/ems.conf
├── sql/                    # 当次上线执行的 SQL (含回滚脚本)
├── backup/                 # 数据库备份输出目录 (软链至数据盘)
├── compose/               # docker-compose.yml 等编排文件
└── /data/
    ├── dockerdata/        # 容器 bind mount 数据 (现网路径见下注)
    ├── ems/logs           # 后端/Nginx 日志
    └── backup              # 备份与 binlog

```

现网 192.168.8.54 既有数据目录为

/home/inspur/xuhz/docker/dockerdata/ (见 docker-compose-54.yml 挂载), 该脚本**不得修改**, 沿用其目录即可。

3.4 用户与权限

```

# 创建专用运行用户 (加入 docker 组以便管理容器; 生产可改用 sudo 白名单收紧)
groupadd -r ems && useradd -r -g ems -m -d /opt/inspur-ems -s /bin/bash ems
usermod -aG docker ems
chown -R ems:ems /opt/inspur-ems /data/ems
# 后端服务禁止以 root 运行 (容器内镜像默认 root 的, 应在编排中追加 user: "1001:1001" 或由宿主机 umask 控制)

```

- 数据库、Redis、RabbitMQ、EMQX 均为应用自建专用账号, 禁止应用直接使用 root/管理员账号;
- 服务器关闭无关账号, 配置 sudo 审计; SSH 建议密钥登录并限制来源 IP。

3.5 系统级初始化

```

timedatectl set-timezone Asia/Shanghai      # 时区 (签名/回调强依赖时钟一致)
systemctl enable --now chronyd && chronyc makestep  # 时间同步 (时钟漂移会导致签名失败)
systemctl disable --now firewalld 2>/dev/null || true  # 若使用云安全组可关闭本机 firewalld; 否则按第 4 章放行
sysctl -w vm.max_map_count=262144           # 部分中间件 (ES 类) 需要; 预先设置无副作用
echo "* soft nfile 65535; * hard nfile 65535" >> /etc/security/limits.conf

```

4 网络与端口规划

原则：80/443 对外开放；MQTT 端口按设备接入网络开放；数据库/缓存/消息队列/管理平台**仅对内网或本机开放**，通过安全组/防火墙限制来源。

端口	服务	用途	对设备网	对用户网	对运维网	防火墙建议
22	SSH	运维登录	拒绝	拒绝	限制源 IP 放行	仅堡垒机/运维网段
80	Nginx	前端页面 + API 反向代理	—	放行	放行	对外开放
443	Nginx	HTTPS（启用证书后）	—	放行	放行	对外开放
8088	后端 Undertow	REST API（直连调试/健康检查）	拒绝	拒绝	放行	仅内网；公网经 Nginx
3306	MySQL 8.0.35	业务库 autoteems	拒绝	拒绝	放行	仅本机/应用节点
6379	Redis 7.2.3	缓存/分布式锁（db13）	拒绝	拒绝	放行	仅本机/应用节点
5672	RabbitMQ	AMQP	拒绝	拒绝	放行	仅应用节点
15672	RabbitMQ Management	管理平台	拒绝	拒绝	放行	仅运维网段
1883	EMQX	MQTT over TCP	按设备网段放行	拒绝	放行	设备接入必需
8883	EMQX	MQTT over TLS	按需	拒绝	放行	启用 TLS 后开放
8081	EMQX	MQTT	按需	拒绝	放行	默认不开

端口	服务	用途	对设备网	对用户网	对运维网	防火墙建议
		over WebSoc ket				放
8083	EMQX	MQTT over WSS	按需	拒绝	放行	默认不开 放
18083	EMQX Dashboar d	管理台 (admin/ public)	拒绝	拒绝	放行	仅运维网 段
6030	TDengine	原生连接 taosd	拒绝	拒绝	放行	仅应用节 点
6041	TDengine	RESTful (后端 JDBC-RS 用此端 口)	拒绝	拒绝	放行	仅应用节 点
9029	前端 devServer	本地开发	不开通 (仅开发 机)			生产禁用
9100/宿 主 9110	XXL-Job 调度中心	任务调度 控制台	拒绝	拒绝	放行	仅运维网 段
9090/宿 主 8690	Spring Boot Admin	监控控制 台	拒绝	拒绝	放行	仅运维网 段

192.168.8.54 现网宿主映射 (docker-compose-54.yml, 勿改) : 后端
1088:8088、前端 3080:80、XXL-Job 9110:9100、Monitor 8690:9090。

firewalld 放行示例 (CentOS/RHEL) :

```
firewall-cmd --permanent --add-service=ssh
firewall-cmd --permanent --add-service=http
firewall-cmd --permanent --add-service=https
firewall-cmd --permanent --add-port=1883/tcp # 设备接入网段建议
再用 rich-rule 限定源
firewall-cmd --permanent --add-rich-rule='rule family="ipv4" source add
ress="10.0.0.0/8" port port="15672" protocol="tcp" accept'
firewall-cmd --reload && firewall-cmd --list-all
```

连通性验证：在应用节点执行 `nc -vz <IP> <port>` 或 `telnet <IP> <port>` 逐项确认。

5 中间件部署 (Docker Compose)

5.1 部署文件说明

文件	用途	是否允许修改
<code>docker-compose.yml</code> (仓库根目录)	自研全栈中间件：MySQL/Redis/RabbitMQ/EMQX/TDengine，网络 <code>inpsur-ems-network</code> ，具名卷持久化	新环境可复制到 <code>/opt/inpsur-ems/compose/</code> 后按章调整口令
<code>docker-compose-54.yml</code> (仓库根目录)	192.168.8.54 现网生产脚本： RabbitMQ/EMQX/TDengine + 后端/前端/XXL-Job/Monitor，复用已有 <code>legal-mysql</code> 、 <code>shared-redis</code>	禁止修改（硬约束）
<code>docker-compose.full.yml</code> / <code>docker-compose-simple.yml</code>	全量/精简参考编排	仅参考
<code>docker/mysql/init.sql</code> 、 <code>docker/tdengine/init.sql</code>	容器首次初始化挂载脚本	只读参考，问题处理见 5.5

5.2 启动自研全栈中间件（推荐全新环境）

前置：Docker Engine 与 Compose V2 已安装并启动；`docker run hello-world` 正常。

```
# 1) 准备目录 (以仓库根目录或 /opt/inpsur-ems/compose 为工作目录)
cd /opt/inpsur-ems/compose
# 将仓库根目录 docker-compose.yml、docker/ 目录、数据库脚本/ 目录放置到对应相对路径
# compose 内挂载：./数据库脚本:/docker-entryptpoint-initdb.d、./docker/tdengine/init.sql 等
```

2) 拉取镜像 (离线环境改为 `docker load -i` 镜像包)

```
docker compose pull
```

3) 启动全部中间件

```
docker compose up -d
```

4) 查看状态与健康检查 (全部 `Up`、`healthy` 后再继续)

```
docker compose ps
```

```
docker compose stats --no-stream
```

逐服务核对 (与 `docker-compose.yml` 保持一致) :

服务	镜像	关键环境变量	卷	健康检查
inpsur-ems-mysql	mysql:8.0.35	MYSQL_ROOT_PASSWORD=123456、 MYSQL_DATABASE=autoee_em s、 TZ=Asia/Shanghai; 启动参数 utf8mb4 / utf8mb4_unicode_ci / max_connections=1000	inpsur-ems-mysql- data:/var/lib/mysql、./数据库脚本:/docker-entrypoint-initdb.d、./docker/mysql/conf:/etc/mysql/conf.d	mysqladmin ping -uroot -p123456
inpsur-ems-redis	redis:7.2.3-alpine	配置以 ./docker/redis/redis.conf 为准 (挂载到 /etc/redis/redis.conf)	inpsur-ems-redis- data:/data	redis-cli ping
inpsur-ems-rabbitmq	rabbitmq:3.12-management-alpine	RABBITMQ_DEFAULT_USER=guest、 RABBITMQ_DEFAULT_PASS=guest	inpsur-ems-rabbitmq- data:/var/lib/rabbitmq	rabbitmq- diagnostics ping
inpsur-ems-emqx	emqx/emqx:5.3.2	Dashboard admin/public 、 EMQX_NAME=em	data/log/plugins 三个具名卷	emqx_ctl status

服务	镜像	关键环境变量	卷	健康检查
inpsur-ems-tdengine	tdengine/tdengine:3.2.0.0	qx、 TZ=Asia/Shanghai TAOS_FQDN=inpsur-ems-tdengine、 TAOS_USER=root、 TAOS_PASS=difyai123456	data/log 具名卷 + ./docker/tdengine/init.sql:/docker-entrypoint-initdb.d/init.sql	taos -c show databases

注意：/docker-entrypoint-initdb.d 下的脚本仅在 **MySQL** 数据卷首次初始化时执行一次；存量卷不会重放。生产环境建议数据库统一按第 6 章手工导入，不依赖挂载自动执行。

逐服务验证命令：

MySQL

```
docker exec -it inpsur-ems-mysql mysql -uroot -p123456 \
  -e "SELECT VERSION(); SHOW DATABASES; SHOW VARIABLES LIKE 'character_set_server';"
```

Redis (自研 compose 是否带密码以 redis.conf 为准；shared-infra 形态必须带密码)

```
docker exec -it inpsur-ems-redis redis-cli ping
docker exec -it inpsur-ems-redis redis-cli -a difyai123456 ping      #
返回 PONG
```

RabbitMQ

```
docker exec inpsur-ems-rabbitmq rabbitmq-diagnostics ping
# 浏览器打开 http://服务器IP:15672 , guest/guest (仅同机登录策略, 远程需建用户)
```

EMQX

```
docker exec inpsur-ems-emqx emqx_ctl status
# 浏览器打开 http://服务器IP:18083 , admin/public (首次登录后立即改密)
```

TDengine

```
docker exec inpsur-ems-tdengine taos -u root -pdifyai123456 -s "SHOW DATABASES;"
```

5.3 shared-infra 形态（本地研发/联调实际使用）

本地研发与部分现场复用共享中间件栈 **shared-infra (localhost)**，后端 `--spring.profiles.active=local` 时连接参数如下（配置见根目录 `application-local.yml` 与 `inpsur-ems-admin/src/main/resources/application-local.yml`）：

中间件	连接参数
MySQL	localhost:3306, root / 123456 , 库 <code>autoee_ems</code>
Redis	localhost:6379, 密码 difyai123456
RabbitMQ	localhost:5672, 用户 rabbitmq / 密码 rabbitmqpassword , vhost = admin_vhost
EMQX(MQTT)	tcp://localhost:1883, admin / public
TDengine	本地联调默认禁用：-- <code>spring.datasource.dynamic.datasource.td.enabled=false</code>

shared-infra 启动与验证：

```
cd <shared-infra 目录>
docker compose --profile core --profile coldchain up -d      # 需要 IoT
时再加 --profile iot
docker exec shared-mysql mysql -uroot -p123456 -e "SHOW DATABASES;"
docker exec shared-redis redis-cli -a difyai123456 ping
docker exec shared-rabbitmq rabbitmqctl status
```

详细整合步骤见 20260911_EMS 与 shared-infra 整合部署说明。

5.4 中间件账号初始化（生产必做）

```
# MySQL: 建应用账号 (替换强口令 EmS#2026!Db 为现场口令), 收回远程 root
docker exec -it inpsur-ems-mysql mysql -uroot -p123456 -e "
CREATE USER 'ems_app'@'%' IDENTIFIED WITH mysql_native_password BY 'EmS
#2026!Db';
GRANT ALL PRIVILEGES ON autoee_ems.* TO 'ems_app'@'%';
FLUSH PRIVILEGES;"
```

```
# RabbitMQ: 建业务用户与 vhost (不要使用 guest 远程接入)
docker exec inpsur-ems-rabbitmq rabbitmqctl add_vhost admin_vhost
docker exec inpsur-ems-rabbitmq rabbitmqctl add_user ems_mq 'EmS#2026!M
q'
docker exec inpsur-ems-rabbitmq rabbitmqctl set_permissions -p admin_vh
```

```
ost ems_mq '.*' '.*' '.*'
docker exec inpsur-ems-rabbitmq rabbitmqctl set_user_tags ems_mq management
```

EMQX: 设备账号统一在 Dashboard 「访问控制 → 认证」中用内置数据库创建 (见 10.1),

Dashboard 管理员口令首次登录 <http://IP:18083> 后强制修改

Redis 生产口令在 docker/redis/redis.conf 中设置 requirepass (若使用 shared-redis 容器, 须以 --requirepass difyai123456 启动, 否则后端认证报错), 并同步修改后端配置。

5.5 TDengine 部署与手动初始化

5.5.1 使用内置 3.2.0.0 (默认)

docker-compose.yml 已声明镜像 tdengine/tdengine:3.2.0.0、root 口令来自 TAOS_PASS=difyai123456、端口 6030/6041。首次启动后手动确认 energy 库与 energy 用户 (详见 TDengine 手动初始化操作手册) :

注意 -p 与密码之间无空格

```
docker exec inpsur-ems-tdengine taos -u root -pdifyai123456 -s "CREATE
DATABASE IF NOT EXISTS energy;"
docker exec inpsur-ems-tdengine taos -u root -pdifyai123456 -s "CREATE
USER energy PASS 'difyai123456';"
docker exec inpsur-ems-tdengine taos -u root -pdifyai123456 -s "GRANT A
LL ON energy TO energy;"
```

验证

```
docker exec inpsur-ems-tdengine taos -u root -pdifyai123456 -s "SHOW DA
TABASES; SHOW USERS;"
docker exec inpsur-ems-tdengine taos -u energy -pdifyai123456 -s "USE e
nergy; SHOW TABLES;"
```

后端启用 TDengine (连接走 RESTful 6041) :

```
java -jar inpsur-ems-admin.jar --spring.profiles.active=prod \
--server.port=8088 --server.servlet.context-path=/autoee-iot-ems \
--spring.datasource.dynamic.datasource.td.enabled=true \
--spring.datasource.dynamic.datasource.td.url='jdbc:TAOS-RS://inpsur-
ems-tdengine:6041/energy?user=energy&password=difyai123456&timezone=Asi
a/Shanghai'
```

5.5.2 升级/使用 TDengine 3.3.6.13 的硬约束 (重点)

1. 密码复杂度: 3.3.6.13 强制口令必须包含大写字母 + 小写字母 + 数字/特殊字符。difyai123456 (纯小写+数字) 不满足复杂度, 必须更换, 例如 Difyai@2026 (示例, 现场自定义并记入密码本)。

2. 不支持 **CREATE USER IF NOT EXISTS**: 3.x 正确语法为 **CREATE USER energy PASS '口令'**; 用户已存在时报 **User already exists**, 跳过即可 (先 **SHOW USERS**; 判断)。
3. 不要使用 **TAOS_ROOT_PASSWORD** 覆盖 **root** 密码: 首次以默认口令登录后, 用 **ALTER USER root PASS '新复杂口令'**; 改密。
4. 不要给容器自定义 **command**: 3.3.6.13 内置 **entrypoint** 已负责初始化与启动, 自定义 **command** (如旧编排里的 **sh -c "sleep 15 && taos -s ... && taosd"**) 会与 **entrypoint** 冲突, 导致初始化异常。
5. 仓库内 **docker/tdengine/init.sql** 使用了 **CREATE USER IF NOT EXISTS ... WITH PASSWORD** 语法, 在 **3.3.6.13** 上会执行失败, 因此 3.3.6.13 一律采用**手动初始化** (容器健康后手工执行 SQL), 不要依赖挂载脚本。
6. 保留字坑: **value** 为 TDengine 保留字, 医院超表数值列已命名为 **metric_value** (见 数据库脚本/**hospital_init_tdengine.sql**); 新增脚本不得使用保留字做列名, 历史对象重命名用 数据库脚本 **/hospital_rename_reserved_words.sql**。

3.3.6.13 手动初始化序列 (示例口令 **Difyai@2026**, 请替换) :

```
docker exec -it inpsur-ems-tdengine taos -u root -p<默认/当前口令>
# taos CLI 内:
ALTER USER root PASS 'Difyai@2026';
CREATE DATABASE IF NOT EXISTS energy KEEP 365 DURATION 10 BUFFER 16;
CREATE USER energy PASS 'Energy@2026';
GRANT ALL ON energy TO energy;
SHOW DATABASES; SHOW USERS;
```

5.6 中间件常用启停

```
docker compose up -d inpsur-ems-mysql          # 单启
docker compose restart inpsur-ems-emqx
docker compose stop inpsur-ems-tdengine && docker compose rm -f inpsur-ems-tdengine
docker compose down                             # 停止并移除容器 (不删具名卷, 数据保留)
docker compose logs -f --tail=200 inpsur-ems-mysql
```

6 数据库部署与初始化 (MySQL)

6.1 建库与字符集

ems_full.sql 已含建库语句 (utf8mb4 / utf8mb4_unicode_ci) , 全新库可直接导入; 手工建库语句:

```
CREATE DATABASE IF NOT EXISTS autoee_ems
  DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE autoee_ems;
```

6.2 脚本清单与导入顺序

顺序	脚本	内容	必选
1	.dev/dumps/ems_full.sql	全量结构 + 基础数据 (MySQL 8.0 导出, 含 CREATE DATABASE) ; 表缺失报 500 时以此补 DDL	全新库必选 (全量)
2	docker/mysql/init.sql	补充表 (如 sys_oss_config 等) 的 CREATE TABLE IF NOT EXISTS	按校验结果执行
3	数据库脚本 /hospital_init.sql	医院模块新增表 (加性, IF NOT EXISTS)	医院模块必选
4	数据库脚本 /hospital_menu.sql	医院菜单 (m2/m3/m4 为后续迭代增量, 按版本依次执行)	必选
5	数据库脚本 /hospital_menu_m2.sql → _m3.sql → _m4.sql	菜单增量 (仅执行目标版本包含的迭代)	按版本
6	数据库脚本 /hospital_role_dashboard.sql、hospital_alarm.sql、	角色看板/告警/修复脚本	按版本说明

顺序	脚本	内容	必选
	hospital_m4.sql 、fix_*.sql		
7	数据库脚本 /hospital_seed_demo.sql	演示数据	仅演示环境
8	数据库脚本 /hospital_init_tengine.sql	TDengine 超表 hospital_device_data	启用 TDengine 时 执行
-	docker/mysql/xxl_job_tables.sql (及 init_xxl_job_tables.sql、 tables_xxl_job.sql)	XXL-Job 调度库 xxl_job	部署 XXL-Job 时

注：数据库脚本/init.sql 当前为空文件，不承担初始化内容；以
docker/mysql/init.sql 与 ems_full.sql 为准。

导入命令（在服务器执行，不要在 Windows 记事本转码后传文件，保持 UTF-8）：

导入前先备份（空库也记录一次基线）

```
mkdir -p /data/backup/predeploy && cd /data/backup/predeploy
docker exec inpsur-ems-mysql mysqldump -uroot -p123456 \
  --single-transaction --routines --triggers --default-character-set=utf8mb4 \
  autoee_ems > autoee_ems_before_$(date +%F_%H%M).sql
```

顺序导入（-o 后导入 ems_full 时其 USE 语句会自动切库）

```
docker exec -i inpsur-ems-mysql mysql -uroot -p123456 --default-character-set=utf8mb4 \
  < .dev/dumps/ems_full.sql
docker exec -i inpsur-ems-mysql mysql -uroot -p123456 --default-character-set=utf8mb4 autoee_ems \
  < docker/mysql/init.sql
docker exec -i inpsur-ems-mysql mysql -uroot -p123456 --default-character-set=utf8mb4 autoee_ems \
  < 数据库脚本/hospital_init.sql
docker exec -i inpsur-ems-mysql mysql -uroot -p123456 --default-character-set=utf8mb4 autoee_ems \
  < 数据库脚本/hospital_menu.sql
# 依版本继续: hospital_menu_m2/m3/m4.sql、hospital_role_dashboard.sql、hospital_alarm.sql ...
# 演示环境: hospital_seed_demo.sql
```

```
# TDengine (启用时)
docker exec -i inpsur-ems-tdengine taos -u root -pdifyai123456 \
    < 数据库脚本/hospital_init_tdengine.sql
```

6.3 导入后校验 SQL

```
-- 1) 表数量与核心表
SELECT COUNT(*) FROM information_schema.tables WHERE table_schema='autoee_ems';
SHOW TABLES LIKE 'hospital%';
SELECT COUNT(*) FROM sys_user;           -- 初始管理员 admin 存在
SELECT COUNT(*) FROM sys_menu;           -- 医院菜单已导入 (menu_id 1979000000000000001 等)
SELECT menu_id, menu_name FROM sys_menu WHERE menu_id BETWEEN 1979000000000000001 AND 1979000000000000100;
-- 2) 字符集
SELECT DEFAULT_CHARACTER_SET_NAME, DEFAULT_COLLATION_NAME
FROM information_schema.SCHEMATA WHERE SCHEMA_NAME='autoee_ems';
-- 3) 菜单乱码检查 (历史坑: fix_sys_menu_encoding.sql)
SELECT menu_name FROM sys_menu WHERE menu_name LIKE '%?%' OR menu_name
REGEXP 'Ã|Â';
-- 4) XXL-Job 库
SHOW DATABASES LIKE 'xxl_job';
```

应用账号授权 (若未在 5.4 创建) :

```
CREATE USER 'ems_app'@'%' IDENTIFIED WITH mysql_native_password BY 'EmS
#2026!Db';
GRANT SELECT,INSERT,UPDATE,DELETE ON autoee_ems.* TO 'ems_app'@'%;
-- 若应用需要建表/改表 (加性自动 DDL 场景) 再临时授予 CREATE/ALTER, 稳定后收回
FLUSH PRIVILEGES;
```

7 后端部署

7.1 Maven 打包 (按 profile)

```
# 构建机需 JDK 1.8 + Maven 3.8+
java -version           # 确认为 1.8.x
mvn -v
cd <项目根目录>
mvn clean package -DskipTests
# 产物: inpsur-ems-admin/target/inpsur-ems-admin.jar
```

辅助脚本（仓库根目录，供参考）：build_backend.sh（容器内 Maven 打包）、build_full.sh（mvn clean package -DskipTests -B）、Windows 一键镜像构建 build-docker.ps1。

Windows 构建 Docker 镜像（在仓库根目录）

```
.\build-docker.ps1 -Backend      # 产物镜像 inpsur-ems/backend:latest
.\build-docker.ps1 -Frontend    # inpsur-ems/frontend:latest
.\build-docker.ps1 -XxlJob      # inpsur-ems/xxl-job:latest
.\build-docker.ps1 -Monitor     # inpsur-ems/monitor:latest
.\build-docker.ps1 -All         # 全部
```

后端镜像构建等价命令：docker build -t inpsur-ems/backend:latest -f inpsur-ems-admin/Dockerfile .（多阶段：maven:3.9-eclipse-temurin-17 构建、eclipse-temurin:17-jre-alpine 运行；容器内已内置部分 --add-opens，但列项不全，以 7.2 的 10 项为准）。

7.2 JVM 参数与启动命令（JDK 1.8 / JDK17）

标准启动（**JDK 1.8，生产推荐**）：

```
mkdir -p /opt/inpsur-ems/backend/logs
cd /opt/inpsur-ems/backend
java -server -Xms1g -Xmx2g -XX:MetaspaceSize=128m -XX:MaxMetaspaceSize=
256m \
  -Dfile.encoding=UTF-8 -Duser.timezone=Asia/Shanghai \
  -jar inpsur-ems-admin.jar \
  --spring.profiles.active=prod \
  --server.port=8088 \
  --server.servlet.context-path=/autoee-iot-ems \
  --spring.datasource.dynamic.datasource.td.enabled=false
```

生产固定参数（三个必带）：--spring.profiles.active=local|dev|prod、--server.port=8088、--server.servlet.context-path=/autoee-iot-ems；

TDengine 不启用时显式带 --

spring.datasource.dynamic.datasource.td.enabled=false，启用时设 true 并提供 td 连接串。

若只能使用 **JDK 17/21**，必须追加以下 **10 项 --add-opens**（少一项都可能 **500**）：

```
java --add-opens java.base/java.lang=ALL-UNNAMED \
  --add-opens java.base/java.lang.reflect=ALL-UNNAMED \
  --add-opens java.base/java.math=ALL-UNNAMED \
  --add-opens java.base/java.util=ALL-UNNAMED \
  --add-opens java.base/java.util.concurrent=ALL-UNNAMED \
  --add-opens java.base/java.net=ALL-UNNAMED \
  --add-opens java.base/java.io=ALL-UNNAMED \
```

```
--add-opens java.base/java.nio=ALL-UNNAMED \
--add-opens java.base/sun.nio.ch=ALL-UNNAMED \
--add-opens java.base/sun.security.ssl=ALL-UNNAMED \
-Xms1g -Xmx2g \
-jar inpsur-ems-admin.jar \
--spring.profiles.active=prod \
--server.port=8088 \
--server.servlet.context-path=/autoee-iot-ems \
--spring.datasource.dynamic.datasource.td.enabled=false
```

7.3 配置文件策略

- jar 内配置: inpsur-ems-admin/src/main/resources/application.yml (公共)、application-local.yml、application-dev.yml、application-prod.yml、application-docker.yml;
- **不要改 jar 内文件**。在 jar 同级目录放置 application-override.yml (根目录已提供样例 application-override.yml) 或 application-prod.yml, Spring Boot 自动优先加载 jar 外配置;
- 也可用命令行属性覆盖, 例如:

```
--spring.datasource.dynamic.datasource.master.url='jdbc:mysql://10.0.0.11:3306/autoee_ems?useUnicode=true&characterEncoding=utf8&serverTimezone=GMT%2B8&useSSL=false&allowPublicKeyRetrieval=true' \
--spring.datasource.dynamic.datasource.master.username=ems_app \
--spring.datasource.dynamic.datasource.master.password='EmS#2026!Db' \
--spring.redis.host=10.0.0.12 --spring.redis.port=6379 --spring.redis.password='EmS#2026!Redis' --spring.redis.database=13 \
--spring.rabbitmq.host=10.0.0.13 --spring.rabbitmq.username=ems_mq --spring.rabbitmq.password='EmS#2026!Mq' --spring.rabbitmq.virtual-host=admin_vhost \
--mqtt.host=tcp://10.0.0.14:1883 --mqtt.username=admin --mqtt.password=public
```

生产关键开关: spring.datasource.dynamic.p6spy=false (SQL 分析插件有性能损耗, 生产必须关闭); logging.level.com.inpsur-ems 生产调 info/warn。

7.4 方式一: systemd 托管 (裸机 jar, 推荐生产)

/etc/systemd/system/inpsur-ems.service:

[Unit]

```
Description=Backend (inpsur-ems-admin)
After=network-online.target docker.service
Requires=network-online.target
```

[Service]

```
Type=simple
User=ems
```

```

Group=ems
WorkingDirectory=/opt/inpsur-ems/backend
ExecStart=/usr/bin/java -server -Xms1g -Xmx2g -XX:MetaspaceSize=128m -X
X:MaxMetaspaceSize=256m \
    -Dfile.encoding=UTF-8 -Duser.timezone=Asia/Shanghai \
    -jar /opt/inpsur-ems/backend/inpsur-ems-admin.jar \
    --spring.profiles.active=prod \
    --server.port=8088 \
    --server.servlet.context-path=/autoee-iot-ems \
    --spring.datasource.dynamic.datasource.td.enabled=false
SuccessExitStatus=143
Restart=on-failure
RestartSec=10
LimitNOFILE=65535
StandardOutput=append:/data/ems/logs/backend/systemd.out.log
StandardError=append:/data/ems/logs/backend/systemd.err.log

```

[Install]

```
WantedBy=multi-user.target
```

```

systemctl daemon-reload
systemctl enable --now inpsur-ems
systemctl status inpsur-ems
systemctl restart inpsur-ems
systemctl stop inpsur-ems
journalctl -u inpsur-ems -f

```

7.5 方式二： Docker 容器（现网 compose）

现网使用 docker-compose-54.yml（只读使用，禁止编辑）：后端镜像 inpsur-ems/backend:latest，容器名 inpsur-ems-backend，宿主端口 1088→容器 8088，profile=docker，通过 SPRING_*/MQTT_* 环境变量注入连接信息，日志挂载到 /home/inpsur/xuhz/docker/dockerdata/inpsur-ems-backend-logs，健康检查 GET /autoee-iot-ems/actuator/health。

在 docker-compose-54.yml 所在目录

```

docker compose -f docker-compose-54.yml up -d inpsur-ems-backend
docker compose -f docker-compose-54.yml ps inpsur-ems-backend
docker compose -f docker-compose-54.yml logs -f --tail=300 inpsur-ems-b
ackend
docker compose -f docker-compose-54.yml restart inpsur-ems-backend
docker compose -f docker-compose-54.yml stop inpsur-ems-backend

```

新环境若需要类似的容器化编排，可复制一份新文件（如 docker-compose-prod.yml）修改，切勿动 54 脚本。

7.6 日志位置

形态	路径
裸机 jar (logback 默认)	/home/aaprojects/autoe-ems/springboot/logs/ (sys-console.log、sys-info.log、sys-error.log) ; 建议部署时通过 -Dlogging.file.path=/data/ems/logs/backend 或覆盖配置改到数据盘
systemd	/data/ems/logs/backend/systemd.{out,err}.log
容器 (54)	宿主 /home/inspur/xuhz/docker/dockerdata/inspur-ems-backend-logs

日志滚动策略 (logback.xml) : console 保留 1 天、info/error 各保留 5 天, 按天滚动。

7.7 启动验证与健康检查

1) 进程与端口

```
ss -lntp | grep 8088
```

```
ps -ef | grep inspur-ems-admin.jar | grep -v grep
```

2) 健康端点 (actuator 已开放)

```
curl -s http://localhost:8088/autoe-iot-ems/actuator/health
```

期望: {"status":"UP"}

```
curl -s http://localhost:8088/autoe-iot-ems/actuator/info
```

3) 登录接口烟测

```
curl -i http://localhost:8088/autoe-iot-ems/captchaImage
```

期望 HTTP 200, 返回验证码 JSON

4) 启动日志关键字检查

```
grep -E "Started .*Application in|Undertow started on port|Tomcat|ERROR" /data/ems/logs/backend/sys-console.log | tail -50
```

容器形态健康地址: <http://localhost:1088/autoe-iot-ems/actuator/health>。

8 前端部署

8.1 构建产物

```
cd inpsur-admin-ui
# Node 16/18/20; 首次或依赖变更时安装
npm config set registry https://registry.npmmirror.com
npm install --legacy-peer-deps

# Windows
set NODE_OPTIONS=--openssl-legacy-provider
npm run build:prod
# Linux/Mac
NODE_OPTIONS=--openssl-legacy-provider npm run build:prod
# 产物目录: inpsur-admin-ui/dist
```

环境文件: .env.production (VUE_APP_BASE_API=/prod-api)、.env.development (VUE_APP_BASE_API=http://localhost:8088/autoee-iot-ems)。开发态 npm run dev 监听 **9029** (vue.config.js)。

构建经验详见 前端开发发布经验 与 打包操作说明。

8.2 Nginx 托管 (裸机)

```
mkdir -p /opt/inpsur-ems/frontend
# 上传并解压 dist 到 /opt/inpsur-ems/frontend/dist
# 配置 /etc/nginx/conf.d/ems.conf (关键项见 8.3)
nginx -t && systemctl enable --now nginx && systemctl reload nginx
```

8.3 nginx.conf 关键配置要点

仓库 inpsur-admin-ui/nginx.conf 已含完整配置, 裸机部署按下述要点核对 (容器名按实际替换为 upstream 主机):

```
server {
    listen      80;
    server_name _;
    root        /opt/inpsur-ems/frontend/dist;
    index       index.html;
    charset     utf-8;
    access_log  /var/log/nginx/ems_access.log;
    error_log   /var/log/nginx/ems_error.log;

    # 静态资源强缓存
    location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2|ttf|eot)
$ {
        expires 1y; add_header Cache-Control "public, immutable";
    }
}
```

```

# 生产 API: /prod-api/ → 后端 context-path (注意末尾斜杠映射)
location ^~ /prod-api/ {
    proxy_pass http://127.0.0.1:8088/autoee-iot-ems/;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_connect_timeout 60s; proxy_send_timeout 60s; proxy_read_t
imeout 60s;
}

# 直连 context-path (医院回调 /autoee-iot-ems/hospital/callback/data
走此段)
location ^~ /autoee-iot-ems/ {
    proxy_pass http://127.0.0.1:8088/autoee-iot-ems/;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}

# SPA 路由回退 (防止刷新 404)
location / { try_files $uri $uri/ /index.html; }

location /health { access_log off; return 200 "healthy\n"; add_head
er Content-Type text/plain; }
location ~ /\. { deny all; }
}

```

gzip 建议 (构建已用 CompressionPlugin 生成 .gz 预压缩文件, Nginx 可直接启用) :

```

gzip on;
gzip_min_length 1k;
gzip_comp_level 6;
gzip_types text/plain text/css application/json application/javascript
text/xml application/xml application/xml+rss text/javascript image/svg+
xml;
gzip_vary on;
gzip_static on; # 优先返回 dist 中现成的 .gz 文件 (需 nginx 含 http_gzi
p_static_module)

```


8.4 Docker 构建前端镜像

```
cd inpsur-admin-ui
docker build -t inpsur-ems/frontend:latest .
# 多阶段: node:20-bookworm-slim 构建 (内置 NODE_OPTIONS=--openssl-legacy-provider、npm 国内源、npm run build:prod)
# 运行态: nginx:1.28.3-alpine, dist→/usr/share/nginx/html, nginx.conf→/etc/nginx/conf.d/default.conf
```

现网 54: 镜像 inpsur-ems/frontend:latest, 宿主 3080→容器 80, 容器内反代到 <http://inpsur-ems-backend:8088/autoe-iot-ems/>, Nginx 日志挂载到 [.../inpsur-ems-frontend-logs](#)。

8.5 浏览器验证

1. 打开 <http://服务器 IP/> (现网经 3080 时为 <http://服务器 IP:3080/>; 开发态 <http://localhost:9029/>) ;
2. 登录页正常显示「智慧能源监控平台」标题, 验证码加载;
3. 使用 admin / 123456 登录, 进入首页无白屏、无 404;
4. F12 → Network: 接口请求前缀 /prod-api/..., 状态码 200, 无 502/404;
5. 切换中英文 (i18n) 正常; 强刷任意子路由不出现 404 (验证 try_files) 。

9 配套组件部署 (XXL-Job / Spring Boot Admin / 报表)

9.1 XXL-Job 调度中心

- 镜像 inpsur-ems/xxl-job:4.6.0, 容器 inpsur-ems-xxl-job-admin, 现网端口 9110:9100;
- 调度库 MySQL xxl_job: 用 docker/mysql/xxl_job_tables.sql (或 init_xx1_job_tables.sql / tables_xx1_job.sql) 初始化;
- 访问: <http://服务器 IP:9110/xxl-job-admin> (容器内端口 9100) ;
- 后端执行器配置 (application-prod.yml 对应项) : xxl.job.enabled=true、xxl.job.admin-addresses、xxl.job.access-token=default_token (生产务必修改并与调度中心一致)、执行器 appname=xxl-job-executor、端口 9101、日志保留 30 天。

初始化调度库

```
docker exec -i inpsur-ems-mysql mysql -uroot -p123456 < docker/mysql/xxl_job_tables.sql
```

现网启动 (54 脚本已含服务, 禁止改脚本)

```
docker compose -f docker-compose-54.yml up -d inpsur-ems-xxl-job
curl -s http://localhost:9110/xxl-job-admin/actuator/health
```

9.2 Spring Boot Admin 监控

- 镜像 inpsur-ems/monitor:4.6.0, 容器 inpsur-ems-monitor, 现网端口 8690:9090;
- 管理账号 admin / inpsur123 (环境变量 SPRING_BOOT_ADMIN_ADMIN_USERNAME/PASSWORD, 上线改密);
- 后端作为客户端注册 (spring.boot.admin.client.enabled=true, url 指向 http://inpsur-ems-monitor:9090/admin);
- 访问 http://服务器 IP:8690/admin (以实际 context 为准)。

9.3 报表/文件导出

报表导出依赖后端本地临时目录与上传目录, 生产需覆盖 ruoyi.profile (文件根目录) 与 spring.servlet.multipart.location=/cpems/server/temp (容器内路径, 54 已挂载 temp 卷)。裸机部署示例:

```
mkdir -p /data/ems/upload /data/ems/temp
# 覆盖配置中指定:
# ruoyi.profile: /data/ems/upload
# spring.servlet.multipart.location: /data/ems/temp
```

9.4 其他前端形态

inpsur-admin-ui-v3/ (Vue3 + Vite 技术预研, 含独立 Dockerfile/nginx.conf)、inpsur-uniapp/ (移动端 uniapp), 本期生产以前端目录 inpsur-admin-ui/ 为准; 如需发布, 构建方式见各自 README。

10 设备接入联调

10.1 EMQX 账号与主题

1. Dashboard http://IP:18083 (admin/public, 首登改密);
2. 访问控制 → 认证 → 创建 **Password-Based** 认证, **Backend** 选 **Built-in Database**; 在 Users 中为每个设备/分组建账号 (如 device_em001 / 强口令);
3. 访问控制 → 授权: 配置发布/订阅规则, 设备仅允许发布自有主题, 示例:
 - 发布: electric/{clientid}/#、water/{clientid}/#;
 - 后端服务账号可订阅: electric/#、water/#、electric/all、electric/emsCarson;

4. 平台内置主题（来自 simulator 模板 simulator/config/templates.yaml）：
electric/voltage、electric/current、electric/power、
electric/consumption、water/consumption、electric/emsCarson、
electric/all;
5. 禁用匿名接入（认证链创建后 allow_anonymous=false），并在 Dashboard
「监控 → 客户端/订阅」确认后端在线客户端。

10.2 simulator 联调

```
cd simulator
pip install -r requirements.txt
# 编辑 config/config.yaml: host 改为 tcp://服务器IP:1883, username/passw
ord 与 EMQX 账号一致
python main.py                # Windows 可执行 run.bat
# 轻载/中载/重载压测场景见 config.yaml stress_test (light 10 设备 1 条/秒;
heavy 1000 设备 10 条/秒)
```

联调链路验证（一条数据的闭环）：

1. simulator 日志显示 publish 成功、EMQX「监控 → 主题」可见消息计数；
2. 后端日志出现 MQTT 消息接收与入库日志，RabbitMQ 管理台
hospital.device.data.queue（或业务队列）有投递/消费记录；
3. MySQL 业务表出现最新一条记录；启用 TDengine 时 SELECT * FROM
energy.hospital_device_data ORDER BY ts DESC LIMIT 1; 有数据；
4. 前端设备列表/监测页数据刷新，设备状态变在线（离线阈值
hospital.iot.monitor-offline-minutes=30 分钟）。

10.3 医院 IOT 平台回调

- 回调地址（通知 IOT 平台方配置）：
 - 经 Nginx: http://服务器 IP/autoee-iot-
ems/hospital/callback/data
 - 直连后端: http://服务器 IP:8088/autoee-iot-
ems/hospital/callback/data
- 方法 POST, 报文 JSON;
- 鉴权头: X-IOT-Token: hospital-iot-2026（配置项 hospital.iot.auth-
token, 生产更换）;
- 可选签名: X-IOT-Sign: hex(HMAC_SHA256(secret, timestamp + "\n" +
msgId)), 平台侧置 hospital.iot.sign-enabled=true、
hospital.iot.sign-secret=<密钥> 后与 IOT 方同步;

- IP 白名单: hospital.iot.ip-whitelist 配置 IOT 平台出口 IP (逗号/分号分隔, 空=不限制); 注意取 X-Forwarded-For/X-Real-IP 首 IP, Nginx 必须正确传递代理头;
- 后端已在 security.excludes 放行 /hospital/callback/** (无需登录 token);
- MQ 配置: 交换机 hospital.topic.exchange、队列 hospital.device.data.queue、路由键 hospital.device.data。

联调命令:

```
curl -i -X POST http://服务器 IP/autoee-iot-ems/hospital/callback/data \
-H "Content-Type: application/json" \
-H "X-IOT-Token: hospital-iot-2026" \
-d '{"msgId":"SMOKE-0001","deviceCode":"TEST-CT-01","dataPoints":[]}'
# 期望 HTTP 200, 响应 code=200 / data="OK"
```

11 分环境配置差异表

配置项	local (本地研发)	dev (开发/测试服)	prod (生产)
profile	local	dev	prod (容器为 docker)
后端端口/上下文	8088 //autoee-iot-ems	同左	同左 (54 宿主 1088)
MySQL	localhost:3306 root/123456, 库 autoee_ems	服务器 IP, 独立库	专用账号 ems_app + 强口令; 54 复用 legal-mysql (root/root)
Redis	localhost:6379, 密码 difyai123456	服务器 IP, 带密码	强口令; database=13; 54 复用 shared-redis
RabbitMQ	rabbitmq/rabbitmqpassword, vhost admin_vhost	同左 (服务器实例)	ems_mq 专用账号, vhost admin_vhost
MQTT	tcp://localhost:1883 admin/public	服务器 IP	每设备独立账号, 禁匿名, TLS 按需
TDengine	显式关闭	可选开启	按 license/容量开

配置项	local（本地研发）	dev（开发/测试服）	prod（生产）
	td.enabled=false		启，energy 账号复杂口令
p6spy SQL 日志	可开	关闭	必须 false
日志级别	debug	info	info/warn
前端 API	直连 http://localhost:8088/autoee-iot-ems	/prod-api 经 Nginx	/prod-api 经 Nginx + HTTPS
验证码	开	开	开
演示数据开关	可选 seed_demo	不开	不开

12 安全加固

1. **默认口令全部修改**：MySQL root/123456、Redis（difyai123456）、RabbitMQ guest/guest、EMQX admin/public 与 TDengine root 口令、XXL-Job accessToken（default_token）、Spring Boot Admin（admin/inpsur123）、平台 admin/123456、hospital.iot.auth-token（hospital-iot-2026）。
2. **HTTPS**：Nginx 配置证书（443，TLS1.2+），80 跳转 443；MQTT 可用 8883 证书；证书到期纳入监控（提前 30 天告警）。
3. **最小端口**：按第 4 章收敛安全组，3306/6379/5672/6030/6041 不对用户网与设备网开放，15672/18083/9110/8690 仅运维网段。
4. **账号最小权限**：应用不用中间件管理员账号；MySQL 只授 autoee_ems 库权限；RabbitMQ 按 vhost 授权；EMQX 按主题前缀授权；Sa-Token 已配置 Bearer/Authorization 头、15 天有效期（可按需调短）。
5. **备份**：见运维手册数据备份章节；备份文件异机保存并定期恢复演练。
6. **其他**：开启操作日志（sys_oper_log）与登录日志；XSS 过滤已启用；关闭 Swagger 公网访问（生产 springdoc.api-docs.enabled 可按客户要求关闭）；服务器定期补丁；防病毒/主机安全 Agent 按客户规范安装。

13 部署时间计划（示例：单日窗口，T 日）

序号	时间	步骤	负责人	产出/判定
----	----	----	-----	-------

序号	时间	步骤	负责人	产出/判定
1	T-3 日	资源交付、操作系统初始化、Docker 安装、网络策略开通	客户网管/实施	验收单签字
2	T-2 日	镜像/安装包 /jar/dist/SQL 上传, docker load, 离线校验 MD5	实施	镜像列表齐全
3	T-1 日	中间件启动、健康检查、建库导 SQL、应用账号创建、备份基线	实施/DBA	校验 SQL 通过
4	T 日 09:00	变更冻结、停止旧服务（升级场景）、全量备份	运维/DBA	备份文件可用
5	09:30	部署后端 (jar/容器)、覆盖配置、启动	实施	health UP
6	10:00	部署前端 dist/Nginx, reload	实施	登录页正常
7	10:30	XXL- Job/Monitor 部署并注册	实施	执行器在线
8	11:00	MQTT simulator 一条数据闭环 + IOT 回调 curl 联调	实施/设备厂商 /IOT 方	数据到库到页面、回调 200
9	13:30	冒烟测试用例	测试/客户	用例全过

序号	时间	步骤	负责人	产出/判定
		集（见 05-03 文档第 4 章）		
10	15:00	安全加固核对、监控/备份任务验证	运维	清单全勾
11	16:00	培训/交付说明、观察期值班开始	实施	签字验收

全新部署可在 T-1 完成绝大部分工作，T 日仅做联调与验收；升级部署严格在窗口内执行并预留 2 小时回退余量。

14 割接、灰度与回退概述

- **割接策略**：优先选择业务低峰窗口；设备侧先在测试主题/少量设备灰度（EMQX 按 clientid/主题分批切换），观察 30 分钟无积压后全量切换。
- **灰度方式**：
 1. Nginx upstream 双后端（旧版本 + 新版本）权重 9:1 → 5:5 → 全量（注意 Sa-Token 会话与 Redis 共享，两版本须共用同一 Redis/db）；
 2. 或按院区/部门分批放量（前端入口不变，后端按请求头/IP 分流）。
- **回退概述**（详细步骤、触发条件见 05-03 上线检查与回滚方案）：
 - 后端：systemd 切回 .bak 旧 jar 重启；容器切回旧镜像 tag（inpsur-ems/backend:<旧 tag>）；
 - 前端：dist.bak 回滚或镜像切旧 tag，reload Nginx；
 - 数据库：加性脚本为主（IF NOT EXISTS），回退用预先准备的反向脚本；结构破坏性变更用 T-1 全量备份恢复；
 - 消息：RabbitMQ 堆积消息不清理，回退后由旧版本继续消费；必要时按 msgId 对账补偿；
 - TDengine：仅追加时序数据，回退应用版本即可，不回退数据。

15 验收标准

类别	验收项	判定标准
基础环境	中间件 5 组件	docker compose ps 全部

类别	验收项	判定标准
		Up/healthy, 版本与方案一致
后端	服务状态	systemctl/容器 active, health=UP, 无持续 ERROR
前端	页面访问	http://IP/ 可登录, 核心页面无白屏/404/502
数据库	结构与数据	表数量与基线一致, 菜单完整无乱码, 校验 SQL 全通过
设备链路	MQTT 闭环	simulator 数据 1 分钟内到 MySQL (启用时到 TDengine) 并在页面展示
回调	IOT 回调	对端真实/模拟推送返回 200, 白名单/Token 错误时正确拒绝
配套	XXL-Job/Monitor	执行器注册在线、监控可见 JVM/健康信息
安全	加固项	第 12 章清单全部完成并有记录
备份	恢复演练	MySQL 备份可在演练库成功恢复, 应用可读取
文档	交付物	部署实施方案、运维手册、上线检查与回滚方案、账号密码信封签收

附录 A 参考资料（仓库内现有文档，不重复内容）

- 打包部署常见问题
- 生产环境部署配置
- docker-compose 部署配置总览
- 前后端容器分析与构建指南
- TDengine 手动初始化操作手册
- 20260911_EMS 与 shared-infra 整合部署说明

- WSL-Docker 部署清单
- 打包操作说明
- 前端开发发布经验
- 根目录：54-UPGRADE-INSTRUCTIONS.md、docker-compose-54-upgrade-plan.md、upgrade-54.sh、verify-upgrade-54.sh
- 配套文档：05-02 系统运维手册、05-03 上线检查与回滚方案

附录 B 访问入口速查

入口	地址	默认账号
前端	http://服务器 IP/ (开发 9029, 54 宿主 3080)	admin / 123456 (上线改密)
后端 API	http://服务器 IP:8088/autoee-iot-ems (54 为 1088)	-
健康检查	http://服务器 IP:8088/autoee-iot-ems/actuator/health	-
EMQX Dashboard	http://服务器 IP:18083	admin / public (首登改密)
RabbitMQ 管理台	http://服务器 IP:15672	guest / guest (生产建专用账号)
XXL-Job	http://服务器 IP:9110/xxl-job-admin	按调度中心配置
Spring Boot Admin	http://服务器 IP:8690/admin	admin / inpsur123 (改密)